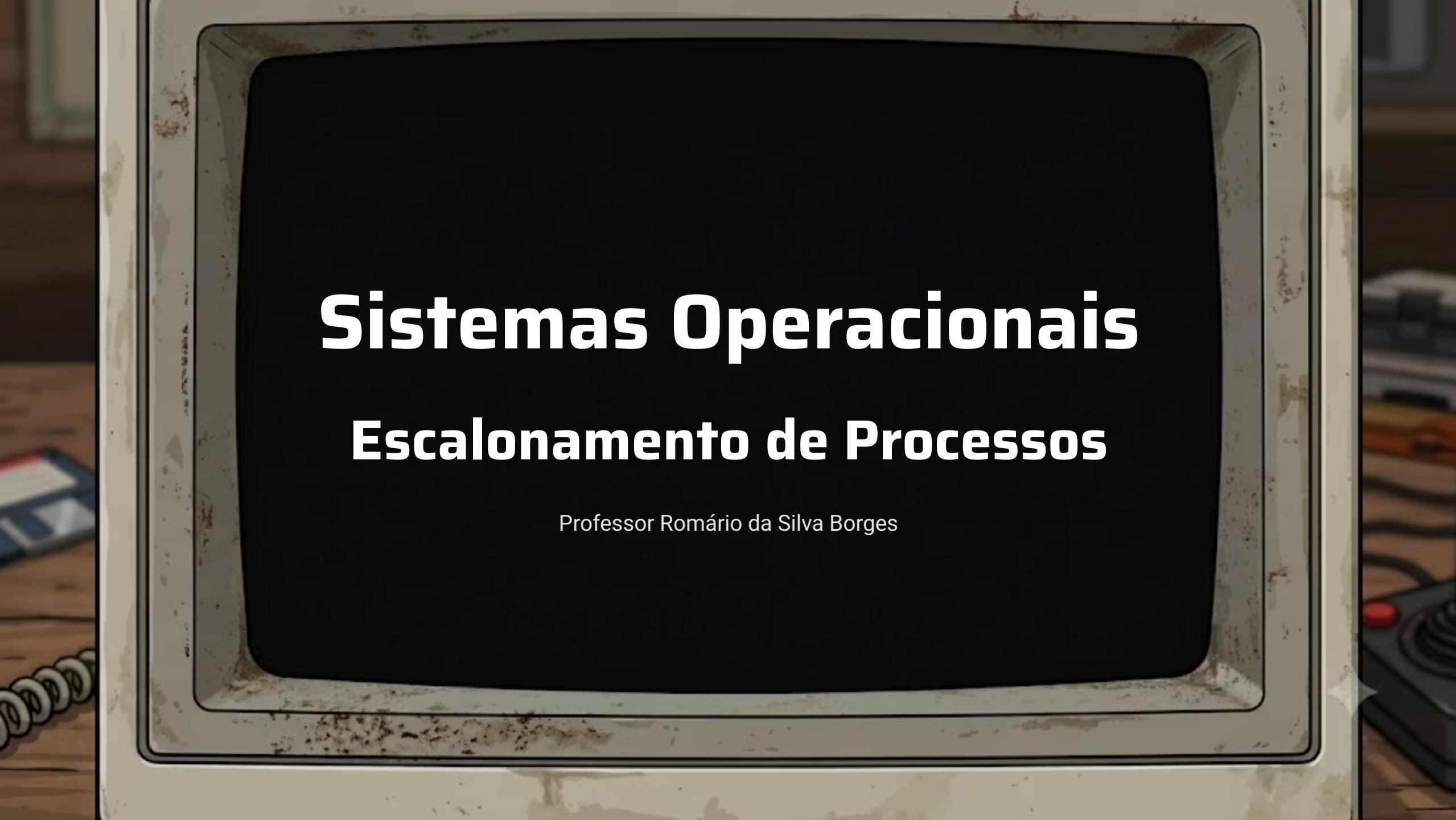




Sistemas Operacionais

Escalonamento de Processos

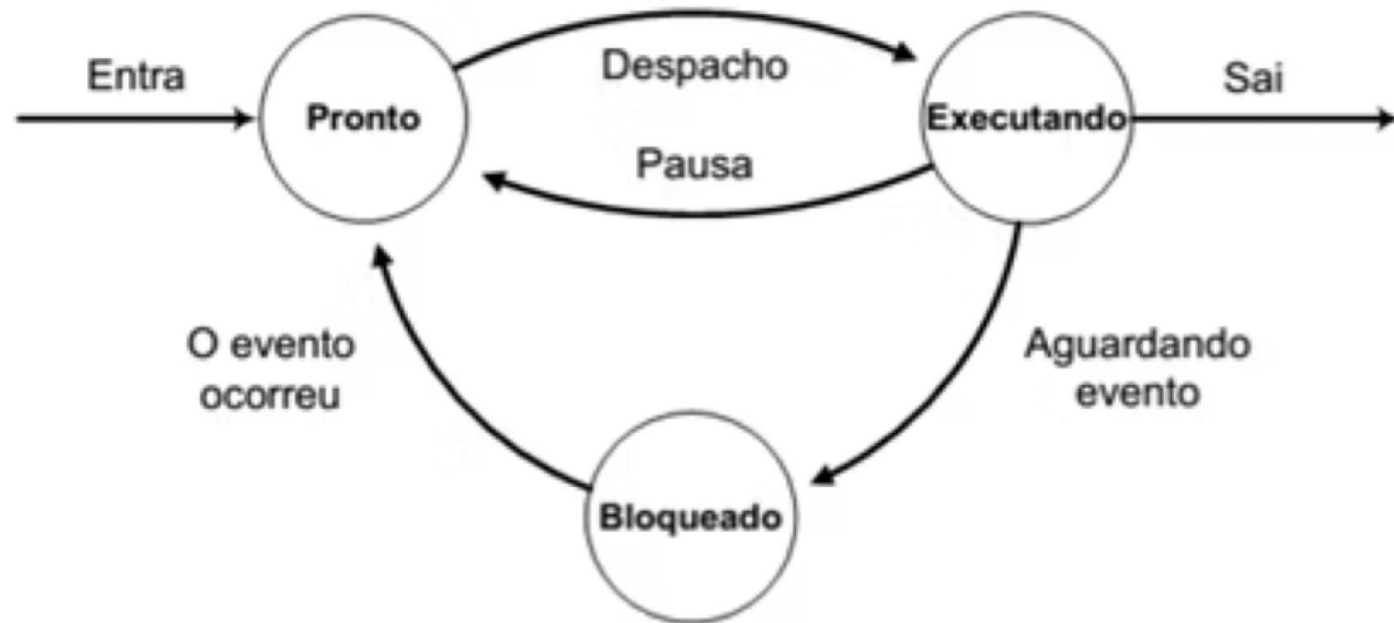
Professor Romário da Silva Borges

Se vários processos estão prontos para executar...

Como o sistema operacional decide quem usa a CPU primeiro?

Nas aulas anteriores...

Estados de um processo:



Fila de Prontos

Fila de prontos

Processos
aguardam na fila



Seleção

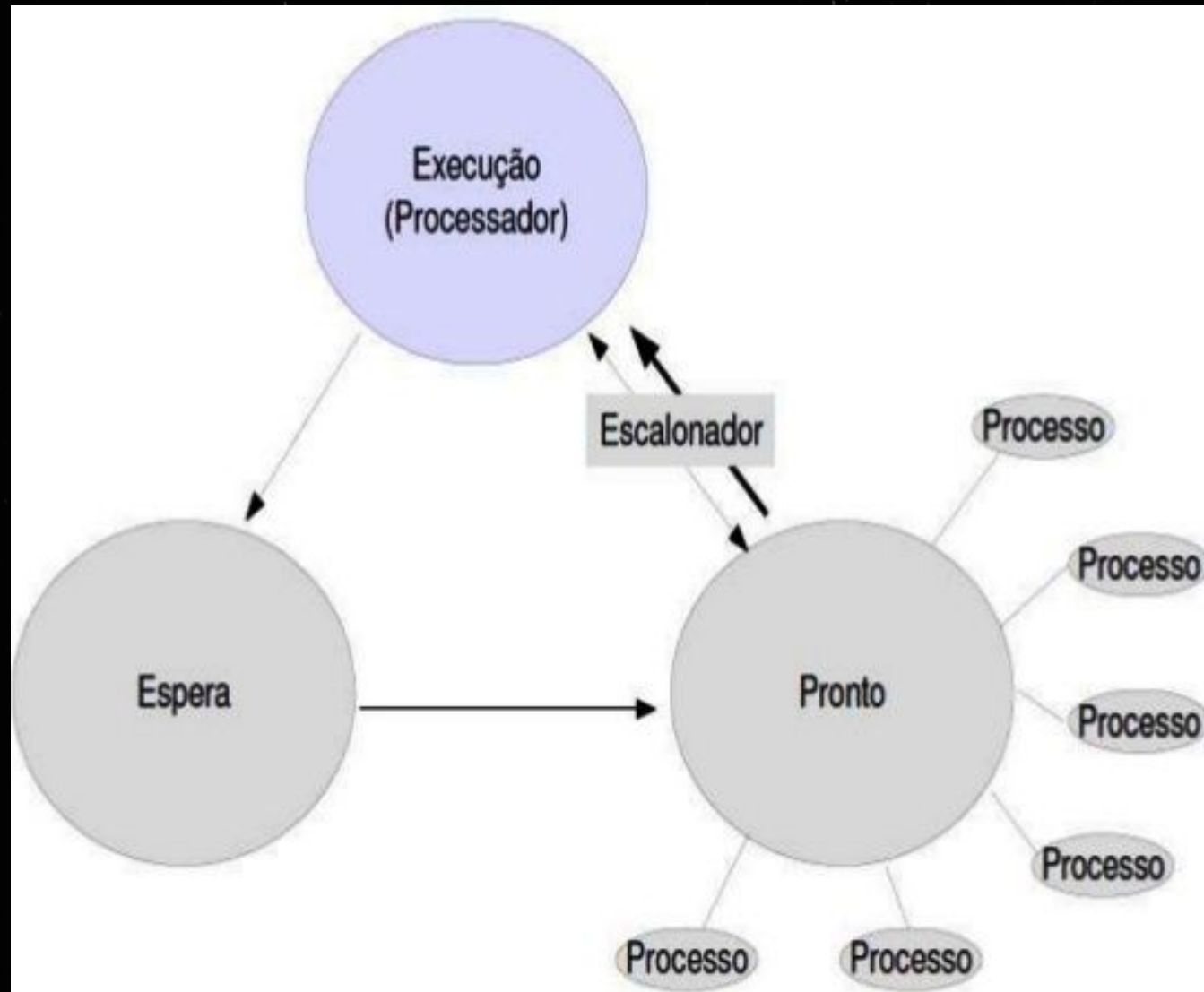
Processo sai da
fila e recebe CPU



Execução

Executa até terminar,
bloquear ou perder vez

Todos os processos em estado **pronto** aguardam na fila **até serem selecionados** para serem executados.



**Qual processo
deve ir primeiro?**

O que é Escalonamento?

Quando ocorre?

Quando **dois ou mais processos** estão no estado **pronto** ao mesmo tempo.

O que o escalonador decide?

- Qual processo deve usar a CPU
- Quando trocar de processo
- Como manter a CPU bem aproveitada

Objetivo

- Garantir justiça entre usuários e processos;
- Utilizar recursos de maneira eficiente;
- Reduzir mudança de contexto;
- Priorizar aplicações;
- Garantir que as políticas sejam cumpridas;

Por que Escalonar era tão Importante?

Sistemas Antigos

- Tempo de CPU era **escasso**
- Algoritmos precisavam balancear bem a carga
- Uma má escolha podia prejudicar muito o desempenho
- Vários usuários ou processos aguardam respostas
- Tempo de resposta tornou-se fator crítico
- Política de escalonamento passou a importar ainda mais

Escalonamento em Computadores Atuais

Computadores Pessoais

- Muitos programas ficam limitados pela **entrada de dados**
- A CPU pode ficar **ociosa** boa parte do tempo
- Alto clock das máquinas influencia a percepção do usuário

Servidores

- Há **muitos serviços** e requisições simultâneas
- O escalonamento continua sendo **muito importante**
- Cenário muito diferente do computador pessoal

Quando Escalonar?



Processo termina

Libera a CPU para outro processo



Processo bloqueia

Aguarda E/S ou evento externo



Retorna à fila

Processo volta ao estado pronto



Interrupção de clock

Relógio sinaliza fim de tempo

Objetivos por Tipo de Sistema

Objetivos por Tipo de Sistema

Sistemas de Tempo Real

Exigem previsibilidade em seus tempos de resposta aos eventos externos, pois geralmente estão associadas ao controle de sistemas críticos, como processos industriais, tratamento de fluxos multimídia, etc;

- Cumprir **prazos** (deadlines)
- Evitar perda de dados
- Garantir **previsibilidade** no comportamento

Objetivos por Tipo de Sistema

Sistemas Interativos

São tarefas que recebem eventos externos (do usuário ou através da rede) e devem respondê-los rapidamente, embora sem os requisitos de previsibilidade das tarefas de tempo real. Esta classe de tarefas inclui a maior parte das aplicações dos sistemas desktop (editores de texto, navegadores Internet, jogos) e dos servidores de rede;

- Reduzir **tempo de resposta**
- Satisfazer **expectativas** dos usuários
- Garantir sensação de fluidez e agilidade

Objetivos por Tipo de Sistema

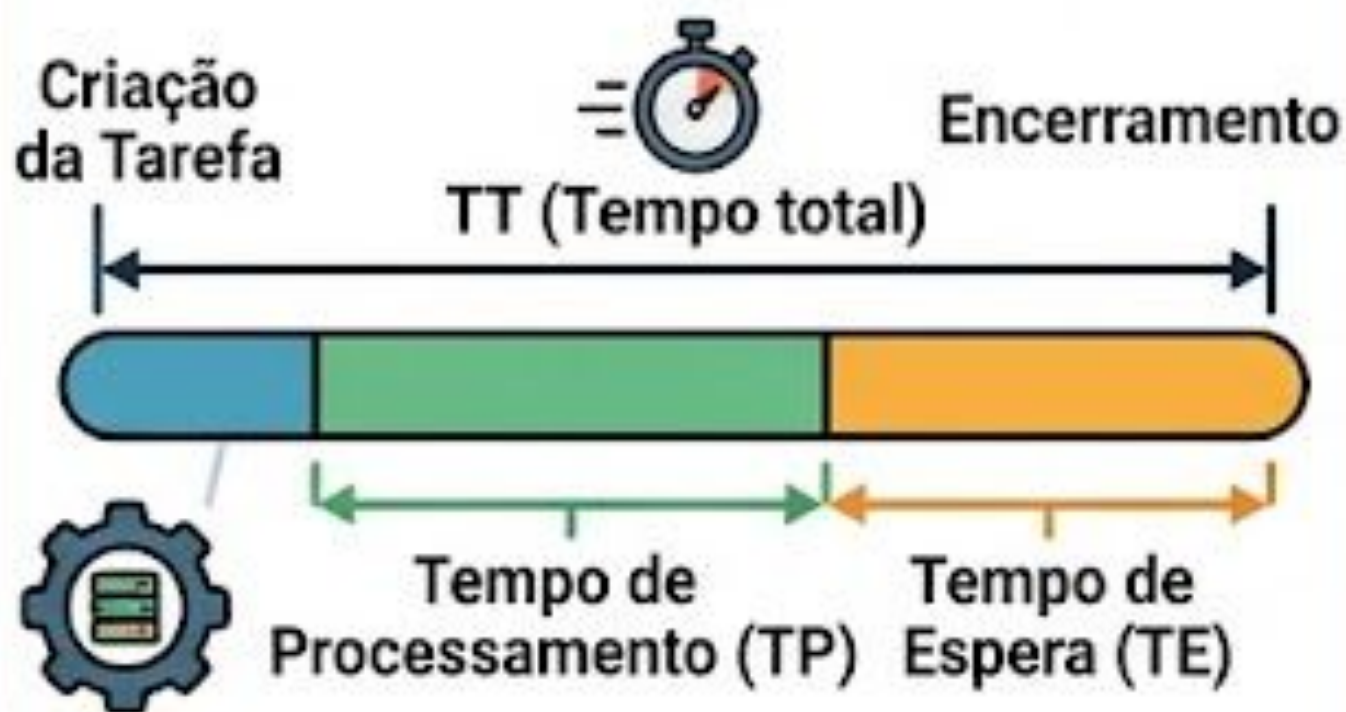
Sistemas em Lote

são tarefas sem requisitos temporais explícitos, que **normalmente executam sem intervenção do usuário**, como procedimentos de backup, varreduras de antivírus, cálculos numéricos longos, tratamentos de grandes massas de dados em lote, renderização de animações, etc.

- Maximizar **vazão** (throughput)
- Reduzir **tempo de retorno**
- Manter a CPU sempre **ocupada**

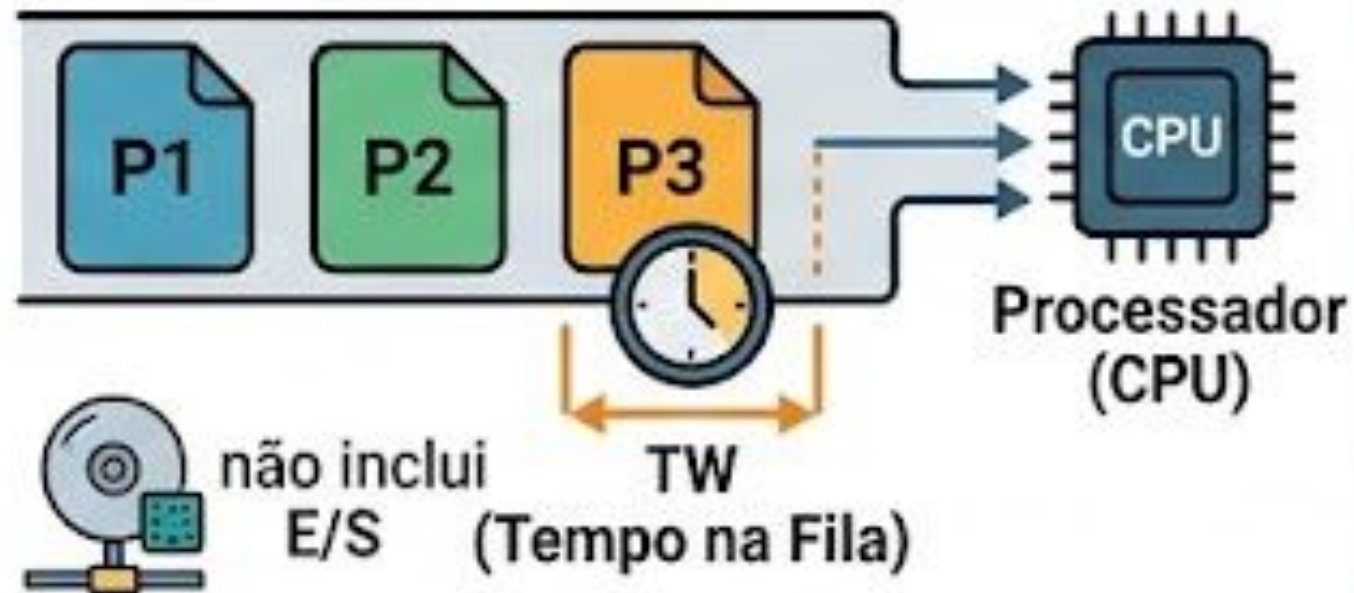
Critérios de Escalonamento

Tempo de Execução (TT - Turnaround Time)

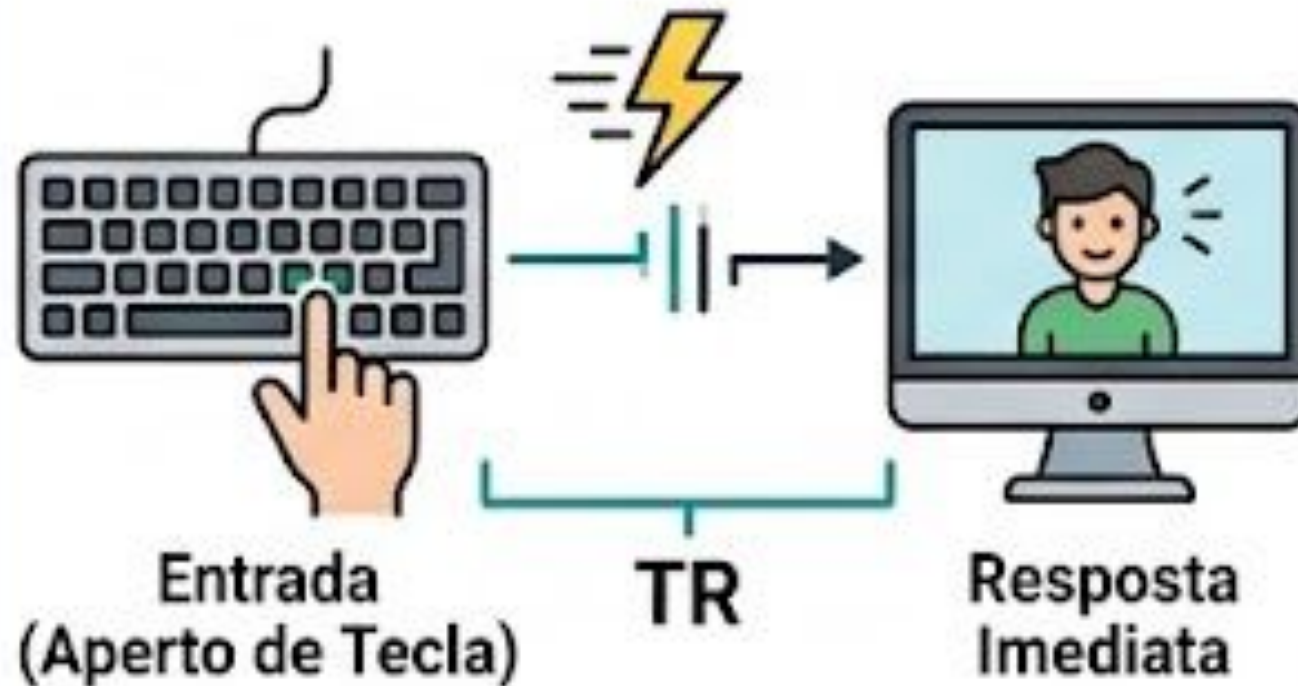


Tempo de Espera (TW - Waiting Time)

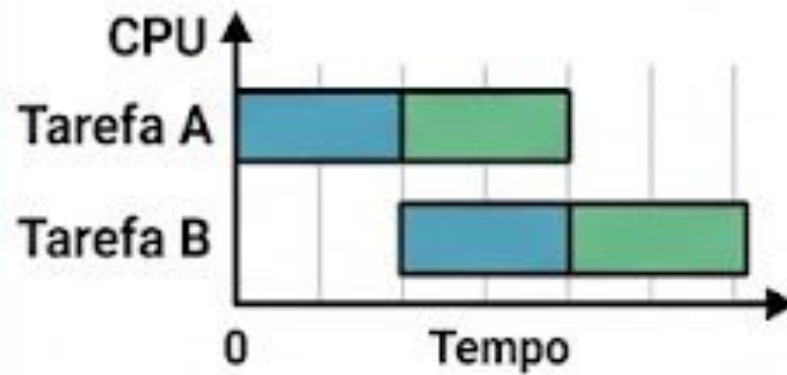
Fila de Tarefas Prontas



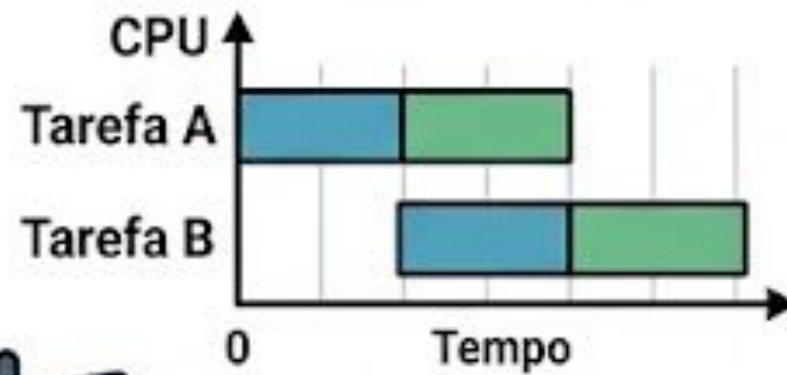
Tempo de Resposta (TR - Response Time)



Justiça

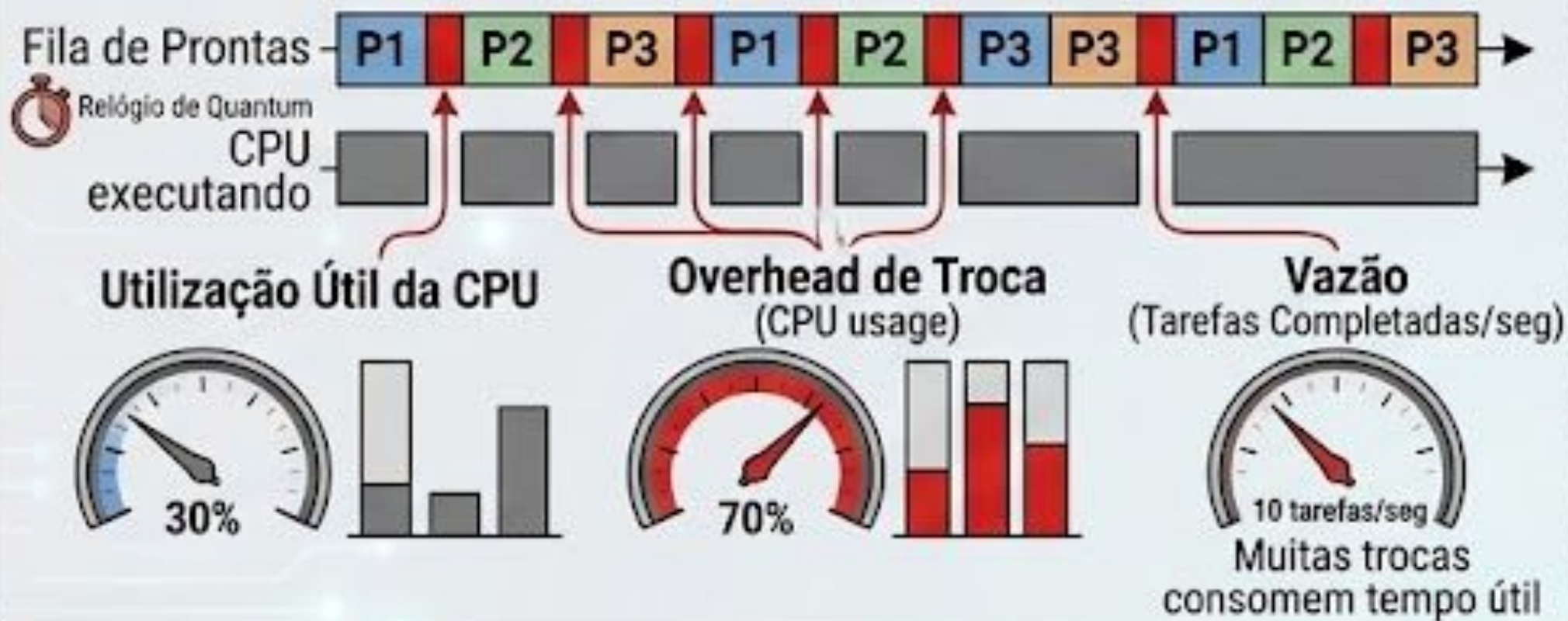


Mesmo Comportamento
e Prioridade

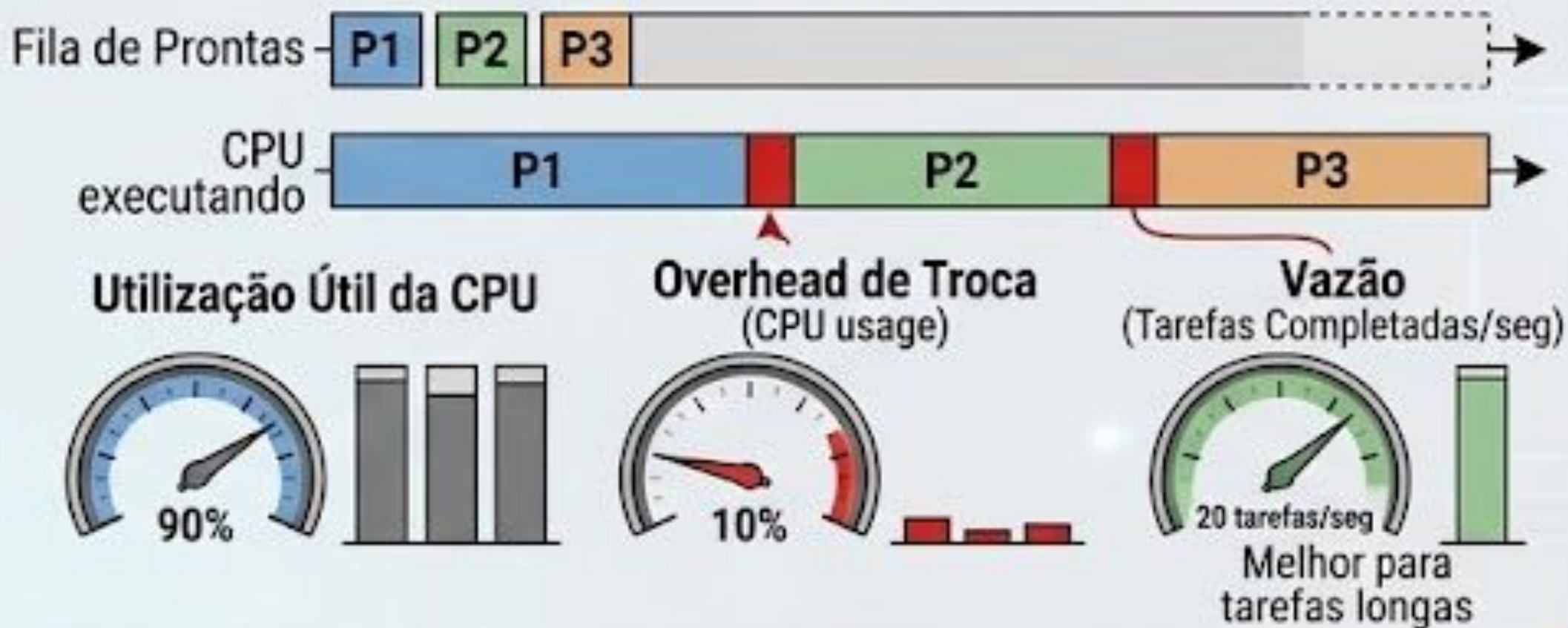


Durações Similares

1A) Cenário: Quantum Curto (5ms) - Alta Interatividade, Baixa Vazão



1B) Cenário: Quantum Longo (50ms) - Baixa Interatividade, Alta Vazão

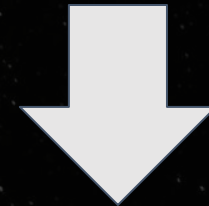


Objetiva-se:

utilização e a vazão da CPU



tempo de execução, o tempo de espera e o tempo de resposta



Procedimento para o Escalonamento

Troca de Contexto

Alternar entre processos tem **custo**. A troca de contexto envolve:

- ① **Salvar registradores**
- ② **Armazenar informações na tabela de processos**
- ③ **Considerar dados de cache**
- ④ **Selecionar novo processo**
- ⑤ **Restaurar informações do novo processo**

Escalonamento Não Preemptivo

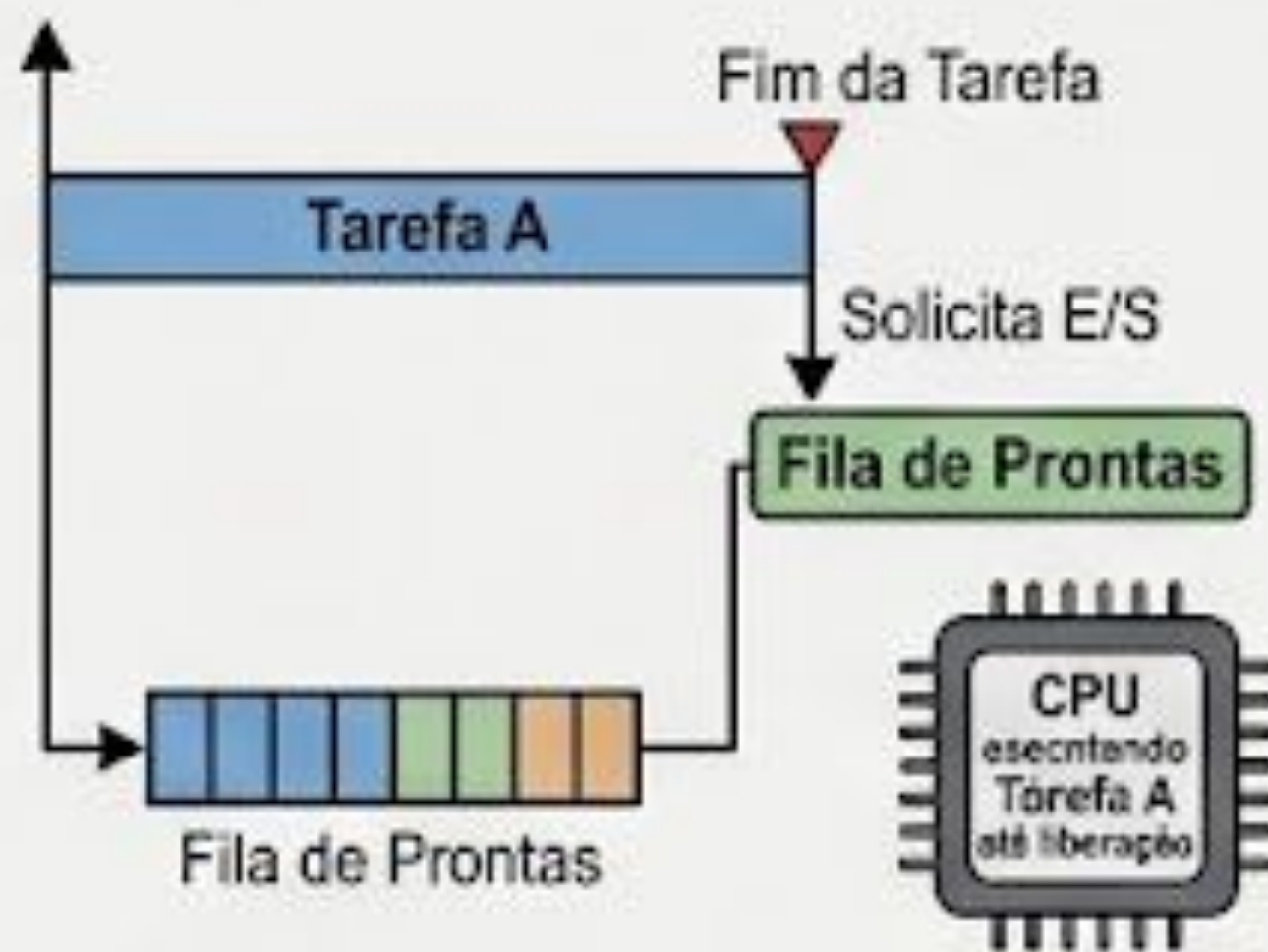
Como funciona?

- O processo executa **até terminar**
- Ou executa **até bloquear**
- Não há decisão baseada em interrupções de relógio

Uso comum

Sistemas em lote — onde não há usuários aguardando resposta imediata.

Sistema Cooperativo



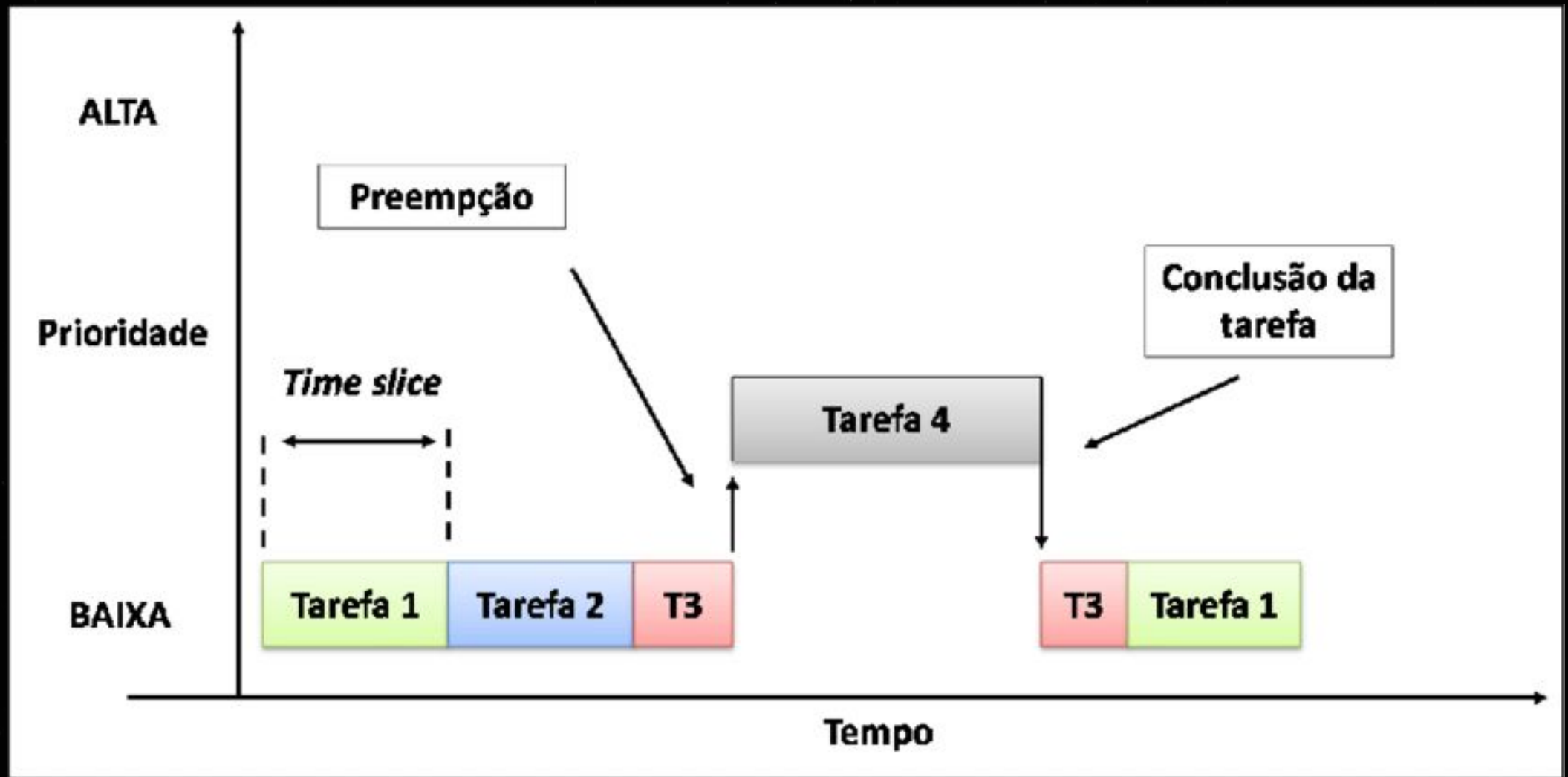
Escalonamento Preemptivo

Como funciona?

- O escalonador pode **interromper o processo**
- A CPU pode ser entregue a outro processo
- O processo também pode sair por bloqueio ou término

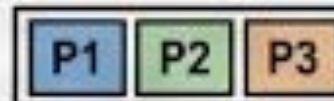
Uso importante

Sistemas interativos — onde usuários esperam resposta rápida.



Quantum de Tempo

Fila de Prontas:

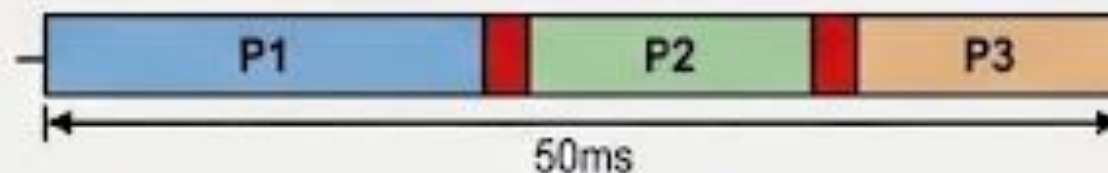


1. Quantum Curto
(5ms)



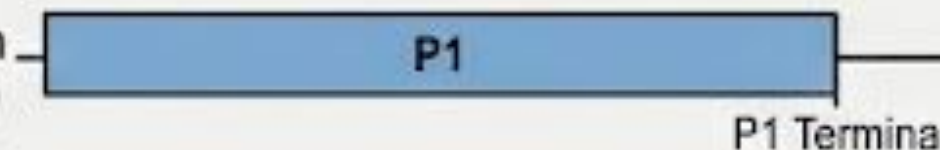
Muitas trocas,
alta interatividade

2. Quantum Longo
(50ms)



Poucas trocas,
baixa interatividade

3. Sem Quantum
(Cooperativo)



Baixa interatividade

Quantum Ideal
Otimizado para eficiência e responsividade

Algoritmos de Escalonamento

FCFS — First Come, First Served

Primeiro a chegar, primeiro a ser servido



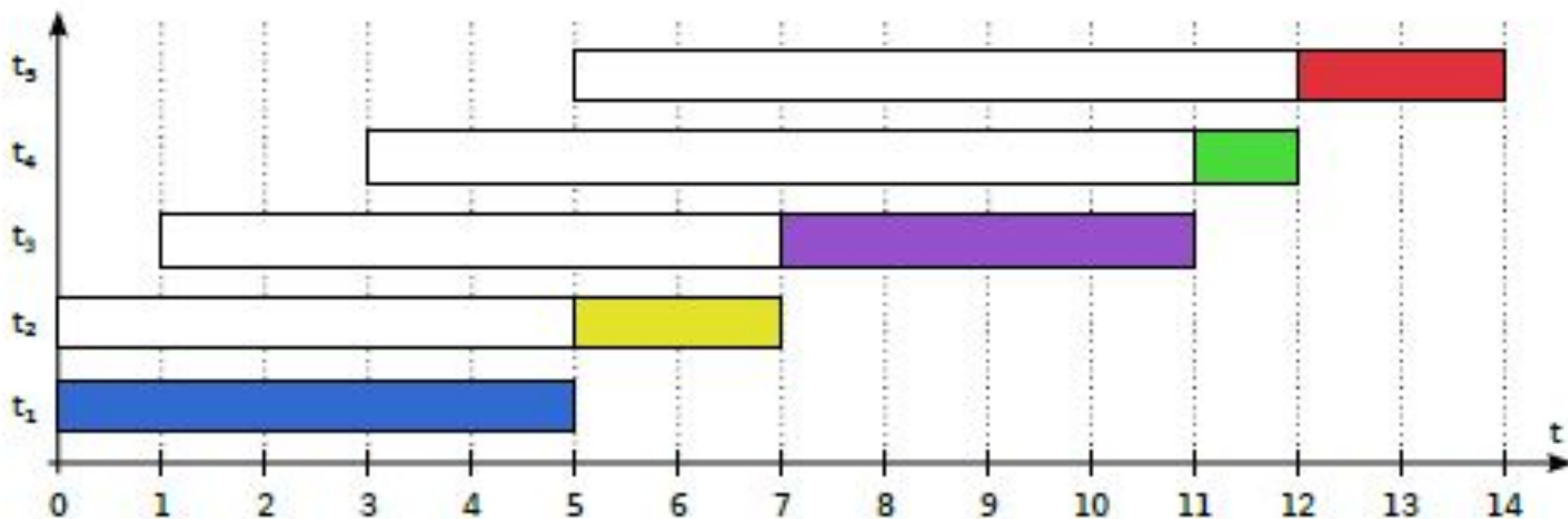


Figura 6.1: Escalonamento FCFS.

Pode ter efeito comboio!

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

FCFS — First Come, First Served

Fácil de entender

Lógica simples e intuitiva

Fácil de implementar

Basta manter uma fila FIFO

Não preemptivo

O processo executa sem interrupções

Limitações do FCFS

Problema do job longo

Se o **primeiro job for demorado**, todos os demais esperam muito na fila.

Jobs curtos prejudicados

Jobs curtos podem ficar **presos atrás de jobs longos**, aumentando o tempo médio de espera.

Round-Robin

Escalonamento por alternância circular



Antigo

Um dos algoritmos mais antigos



Justo

Todos os processos recebem vez



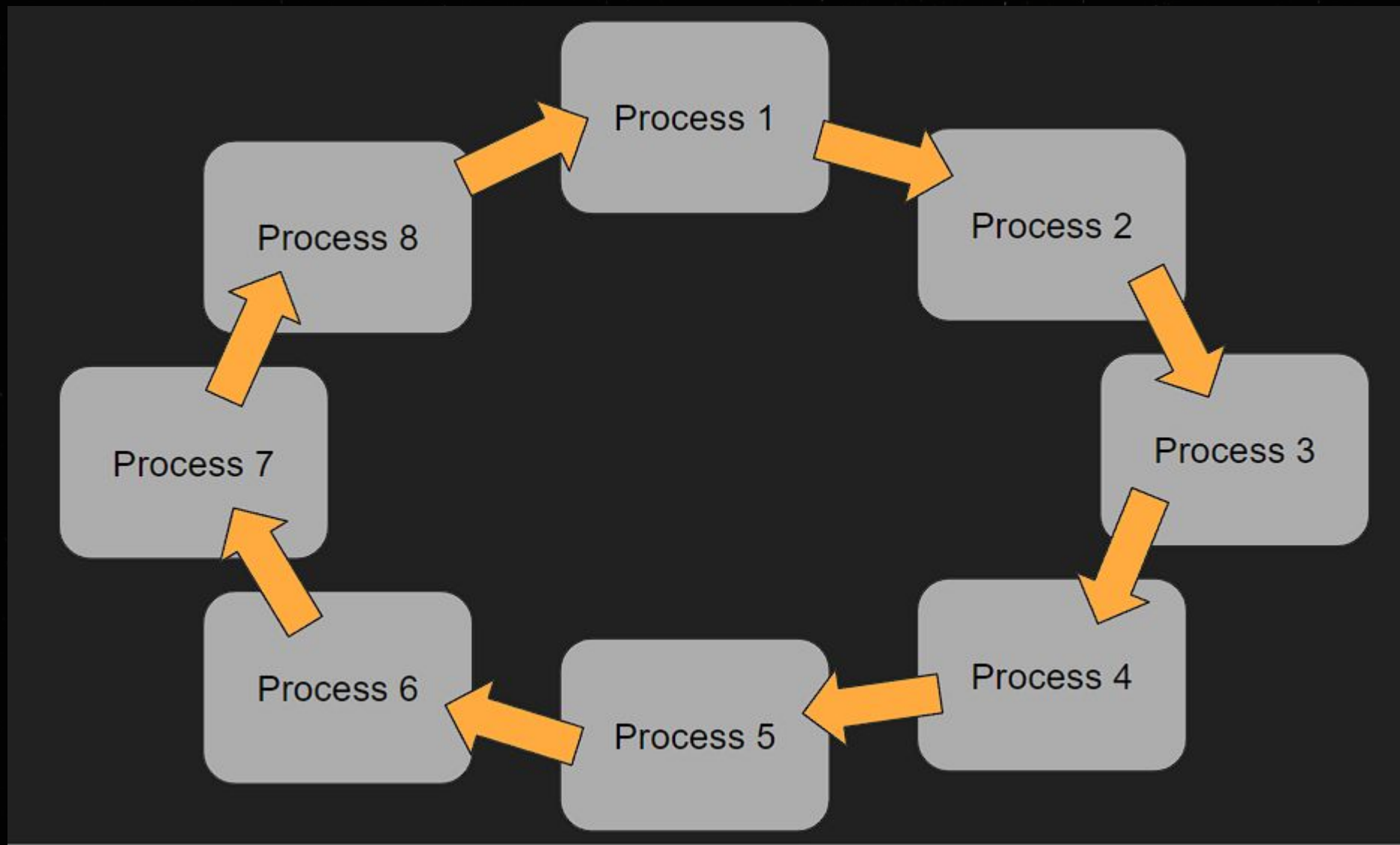
Amplamente utilizado

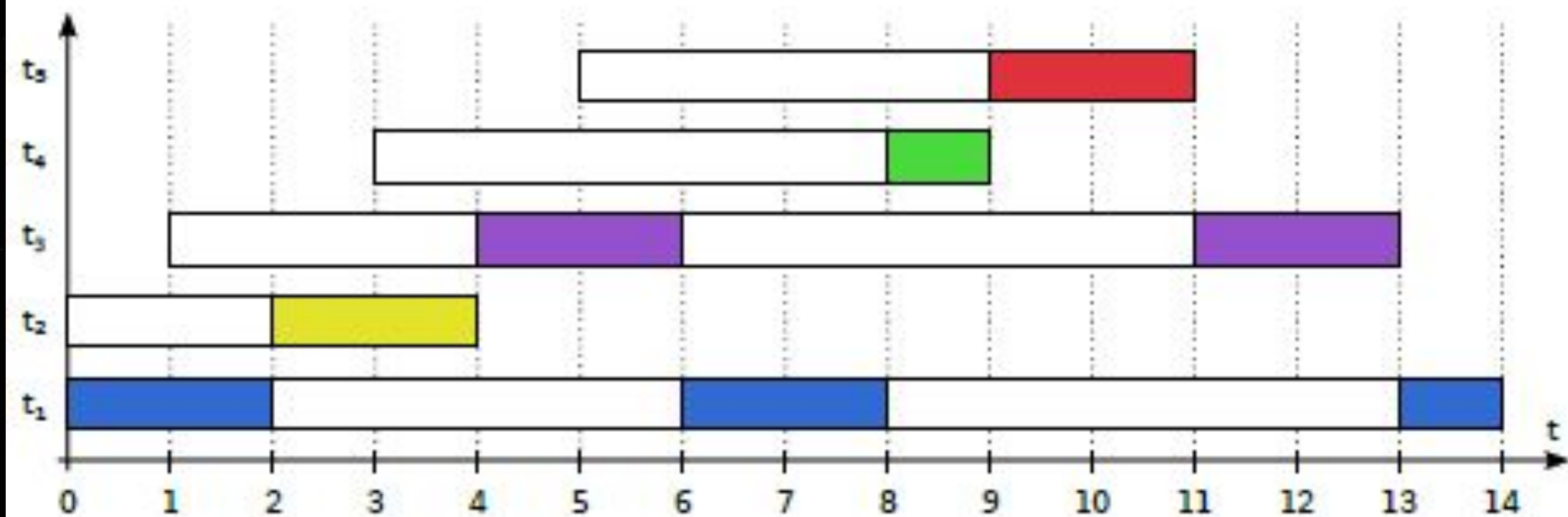
Presente em muitos sistemas



Usa quantum

Tempo máximo por processo





Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

SJF — Shortest Job First

Job mais curto primeiro

Menor tempo total

Executa primeiro o job com
menor tempo estimado

Não preemptivo

O job escolhido executa
sem interrupção

Estimativa necessária

Exige conhecer ou **estimar** o
tempo de processamento

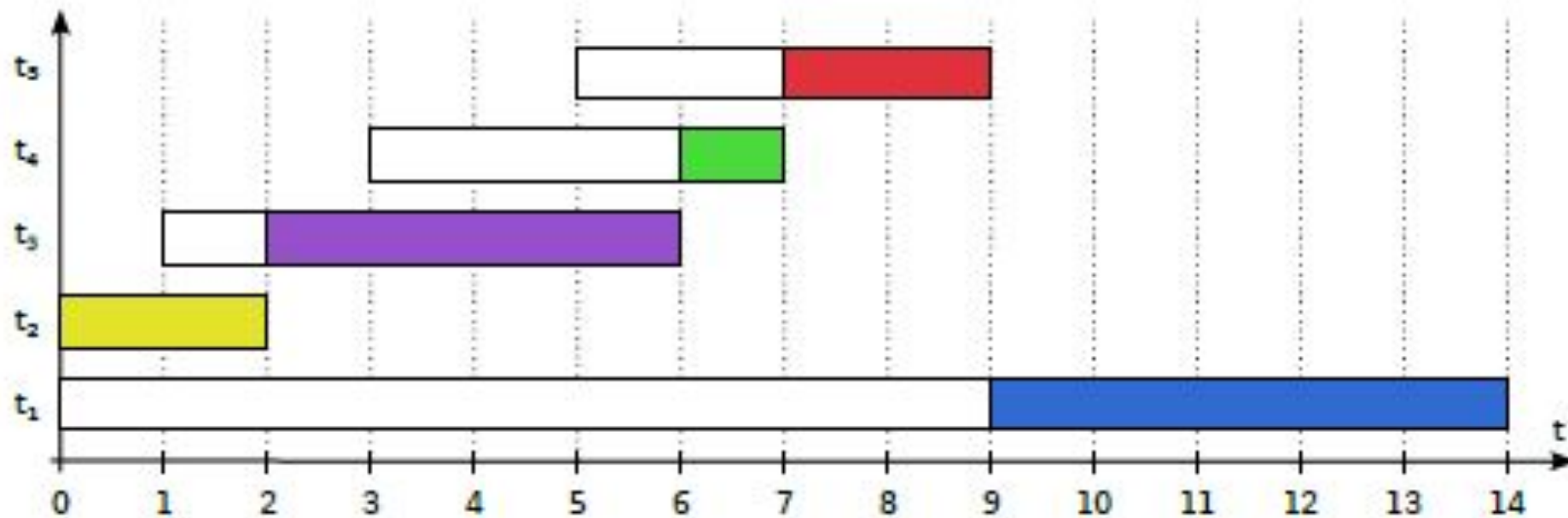


Figura 6.4: Escalonamento SJF.

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

Pode ter inanição (starvation)!

SRTF — Menor Tempo Restante

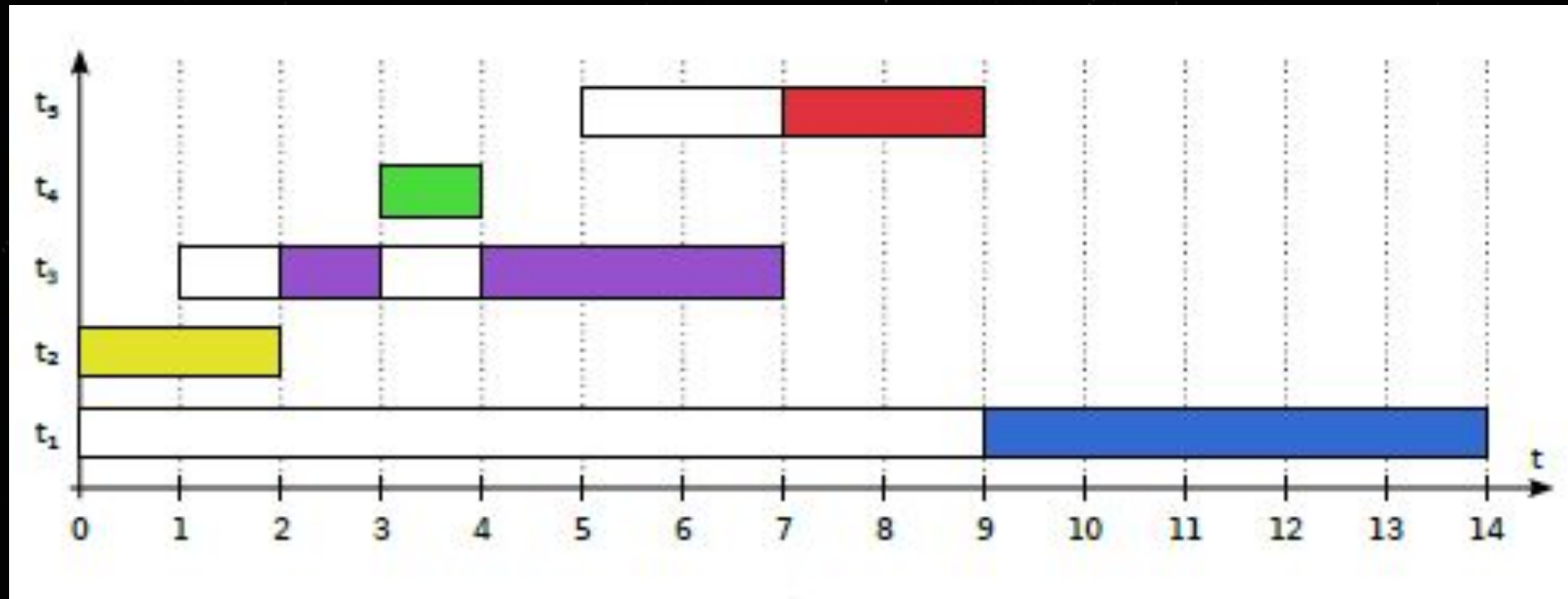
Executa o processo com o **menor tempo restante**.

Semelhante ao SJF

Mesma ideia de priorizar o job mais curto

Mais responsivo

Jobs curtos que chegam tarde não ficam presos na fila



Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

Pode ter inanição (starvation)!

Escalonamento por Prioridades

Processos podem ter **necessidades diferentes** — nem todos merecem o mesmo tratamento. Prioridade pode ser por: data, duração, tipo de tarefa, proprietário...

O escalonamento por prioridade pode ser cooperativo ou preemptivo.

Processos com usuários

Processos que se comunicam com usuários podem receber **mais atenção** do escalonador

Fila de prioridades

A fila não precisa ser FIFO — pode haver **filas com diferentes prioridades**



Qual a política a ser usada
para definir as prioridades?

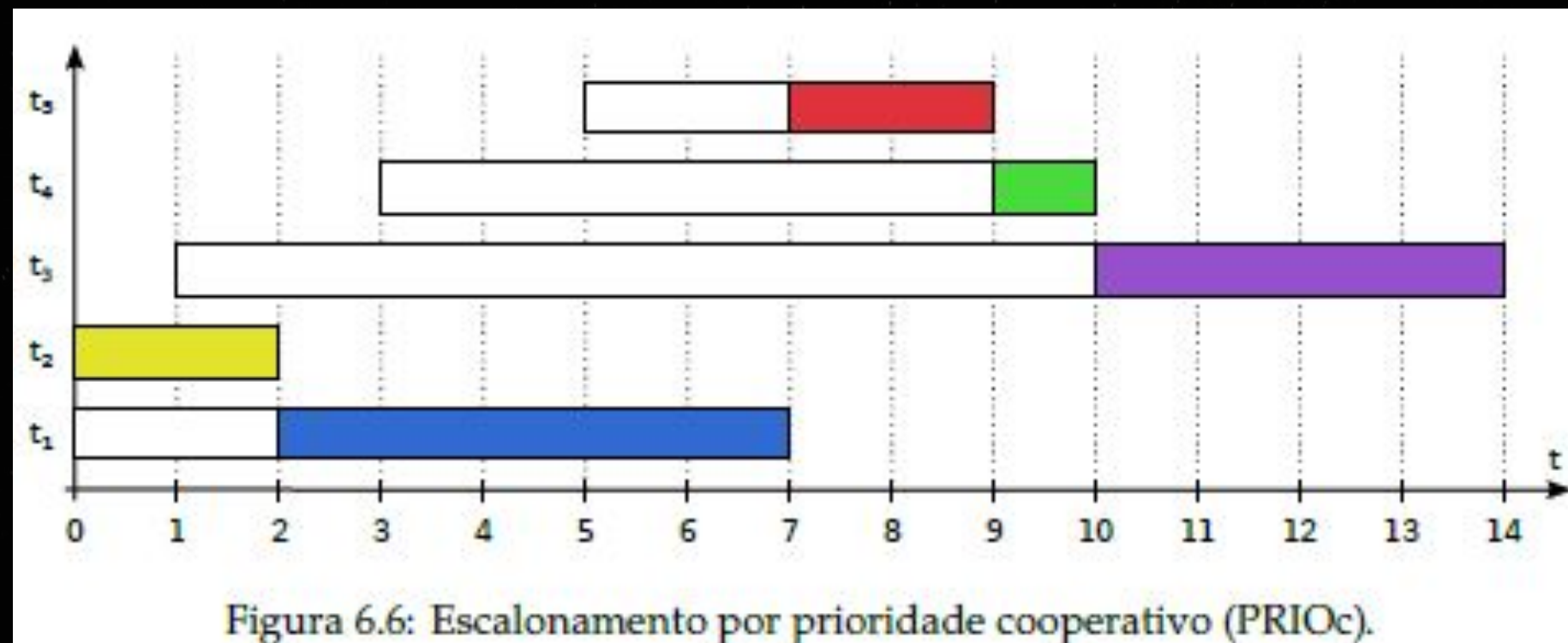
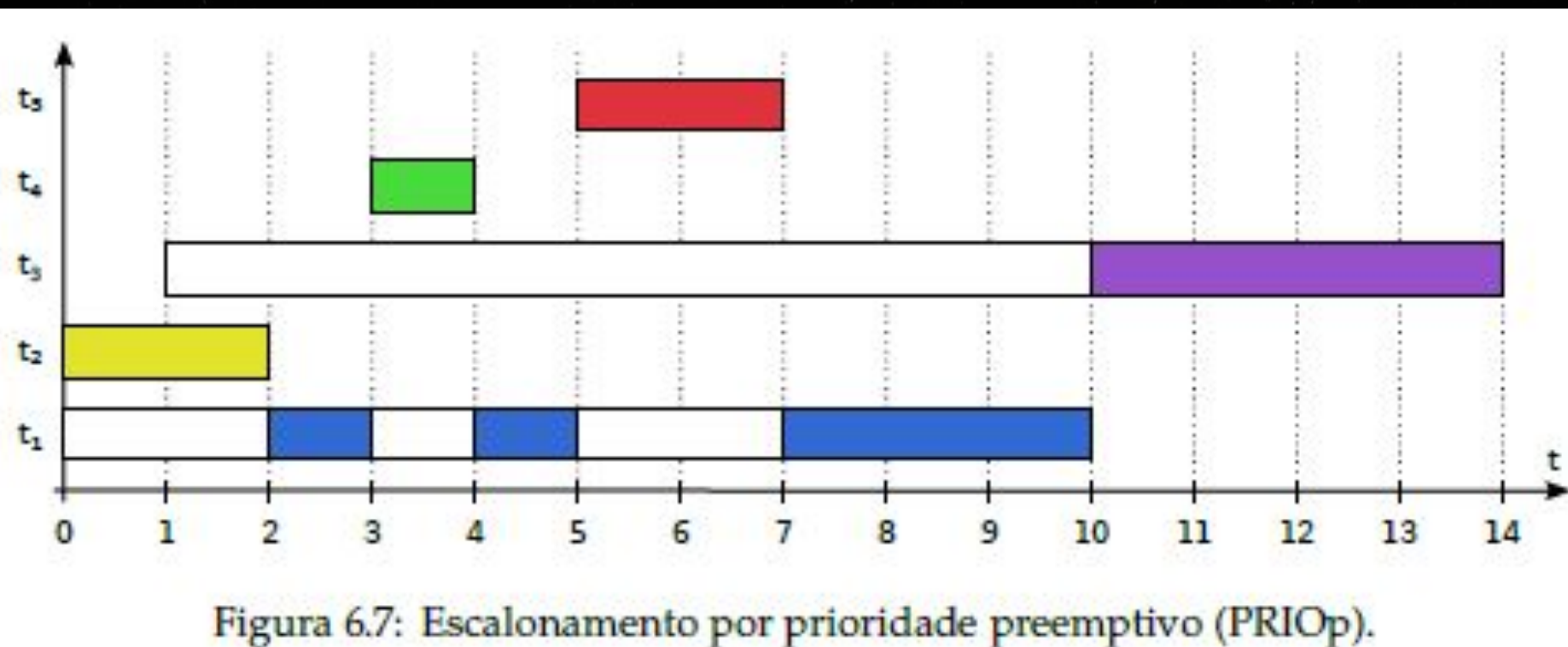


Figura 6.6: Escalonamento por prioridade cooperativo (PRIOc).

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

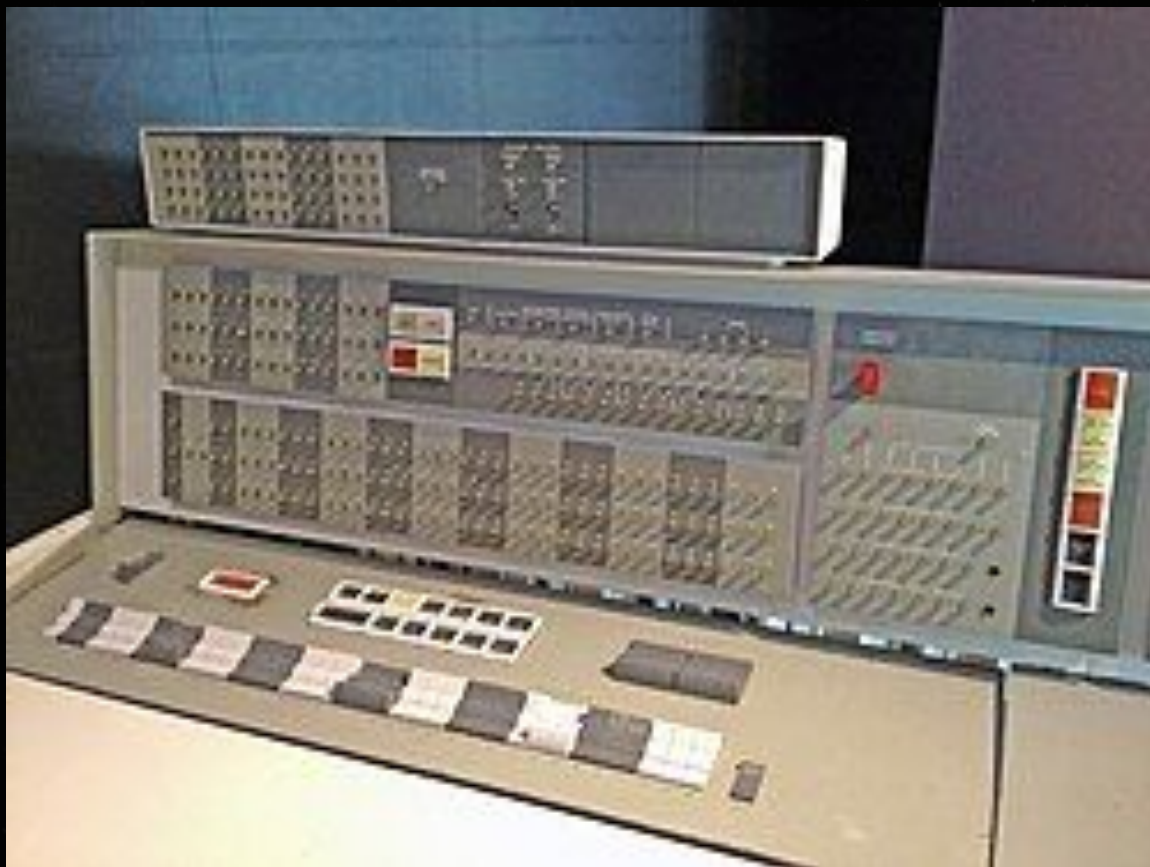


Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.



O que pode acontecer se os
de prioridade alta se
mantiverem em execução?



O que pode acontecer se os
de prioridade alta se
mantiverem em execução?
R: “Envelhecimento”

Processo de baixa prioridade rodava
até o desligamento dele em 1973!

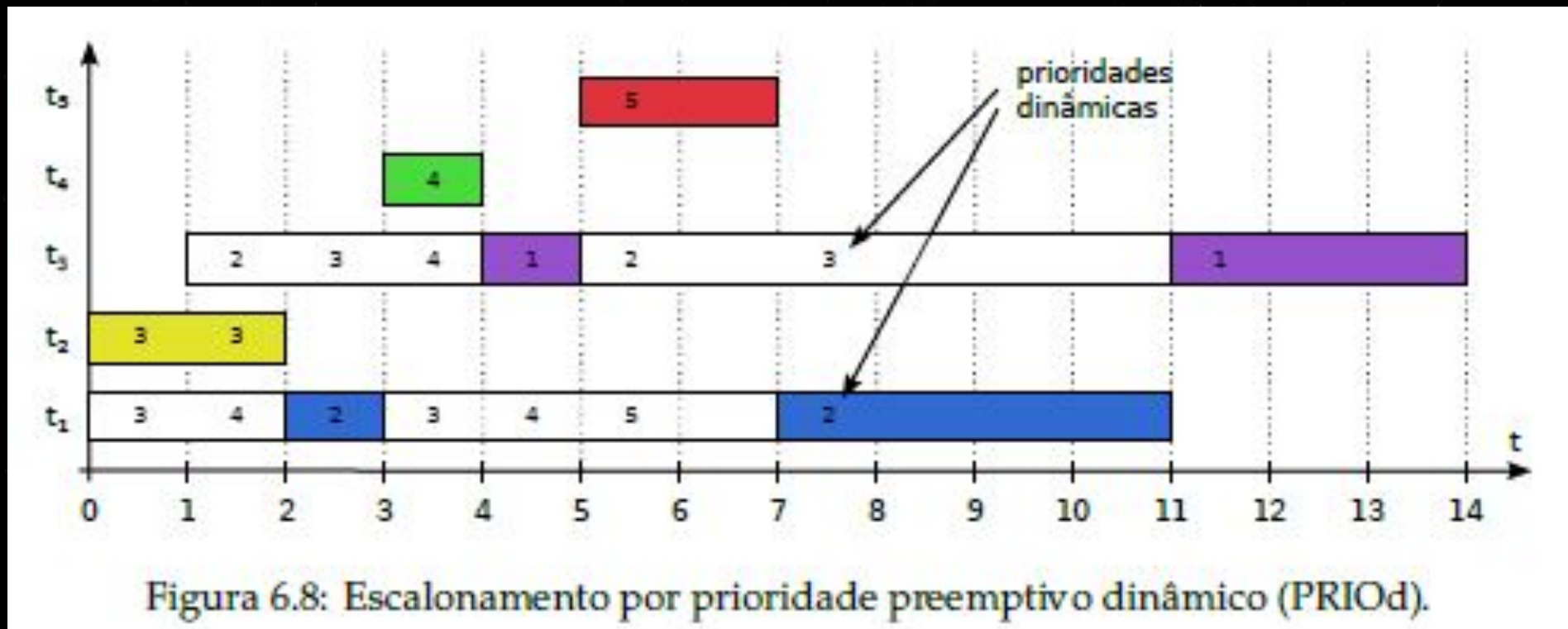


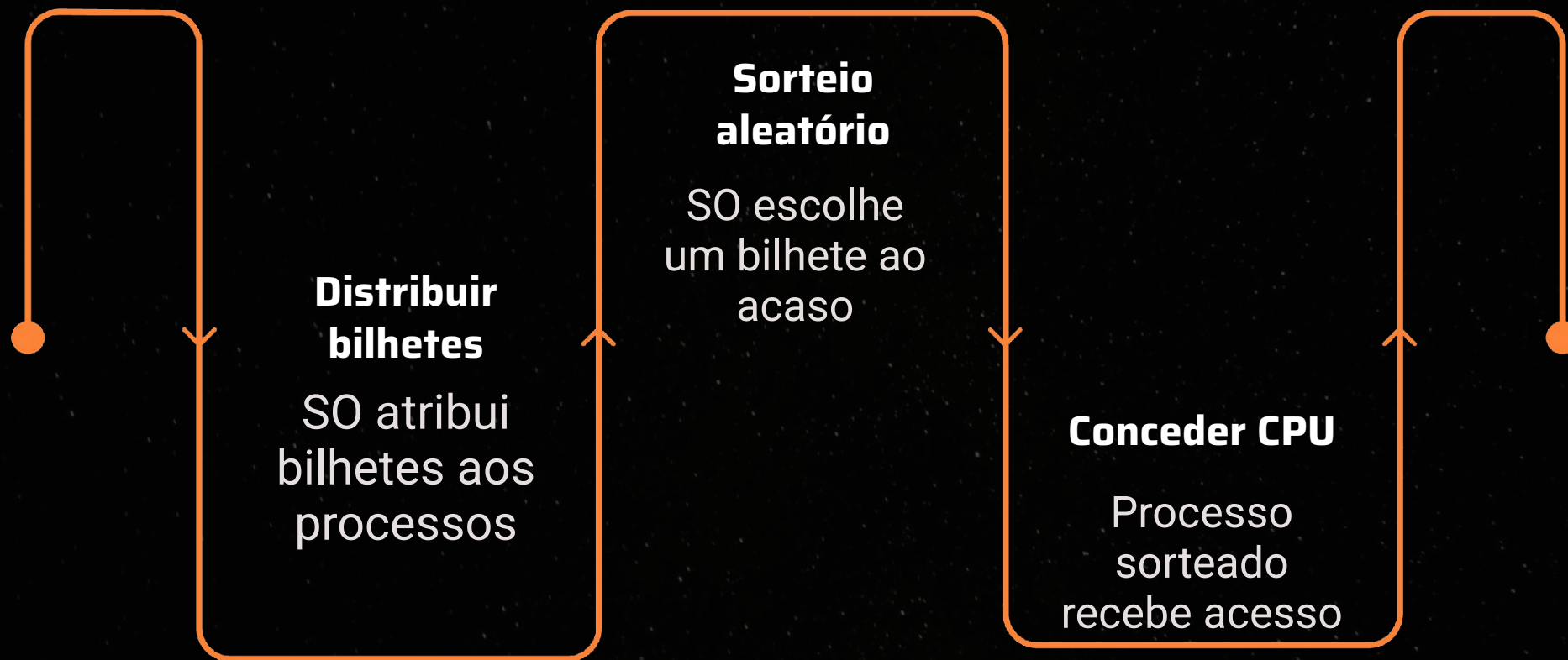
Figura 6.8: Escalonamento por prioridade preemptivo dinâmico (PRIOd).

tarefas “envelhecem” um pouco e no próximo turno terão mais chances de ser escolhidas

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

Tabela 6.1: Tarefas na fila de prontas.

Escalonamento por Loteria



Processos mais importantes recebem **mais bilhetes**, aumentando sua chance de ganhar a CPU.

Loteria: Probabilidade e Limitações

$$P(\text{ganhar}) = \text{bilhetes do processo} / \text{total de bilhetes}$$

⚠ Depende de aleatoriedade

Resultados dependem de **números aleatórios** — pode não ser perfeitamente previsível.

⚠ Starvation possível

Ainda pode ocorrer starvation por muito tempo, embora com **baixa probabilidade**.

Algoritmo de escalonamento	FCFS	RR	SJF	SRTF	PRIOc	PRIOp	PRIOd
Tempo médio de execução T_t	8,0	8,4	5,8	5,4	6,6	5,6	5,8
Tempo médio de espera T_w	5,2	5,6	3,0	2,6	3,8	2,8	3,0
Número de trocas de contexto	4	7	4	5	4	6	6
Tempo total de processamento	14	14	14	14	14	14	14

Tabela 6.2: Comparação entre os algoritmos apresentados.

Escalonamento no Linux

Escalonamento no Linux

No Linux, as tarefas são divididas em diversas classes de escalonamento, de acordo com suas demandas de processamento. Cada classe possui sua própria fila de tarefas, em um esquema conhecido como Multiple Feedback Queues [Arpaci-Dusseau and Arpaci-Dusseau, 2014]. As classes definidas no escalonador atual (núcleo 4.16, em 2018);

Classe SCHED_DEADLINE: classe específica para tarefas de tempo real que devem executar periodicamente e respeitar prazos *deadlines* predefinidos. A fila de tarefas desta classe é organizada por um algoritmo chamado *Earliest Deadline First* (EDF), usual em sistemas de tempo real [Farines et al., 2000].

Classe SCHED_FIFO: tarefas nesta classe são escalonadas usando uma política com prioridade fixa preemptiva, sem envelhecimento nem *quantum*. Portanto, uma tarefa nesta classe executa até bloquear por recursos, liberar explicitamente o processador (através da chamada de sistema `sched_yield()`) ou ser interrompida por outra tarefa de maior prioridade nesta mesma classe.

Classe SCHED_RR: esta classe implementa uma política similar à SCHED_FIFO, com a inclusão da preempção por tempo (*Round-Robin*). O valor do *quantum* é proporcional à prioridade de cada tarefa, variando de 10ms a 200ms.

Classe SCHED_NORMAL ou SCHED_OTHER: é a classe padrão, que suporta as tarefas interativas dos usuários. As tarefas são escalonadas por uma política baseada em prioridades dinâmicas (envelhecimento) e *Round-Robin* com *quantum* variável.

Classe SCHED_BATCH: similar à classe SCHED_NORMAL, mas pressupõe que as tarefas são orientadas a processamento (*CPU-bound*) e portanto são ativadas menos frequentemente que as tarefas em SCHED_NORMAL.

Classe SCHED_IDLE: classe com a menor prioridade de escalonamento; tarefas nesta classe só recebem processador caso não exista nenhuma outra tarefa ativa no sistema.

Multiprocessadores

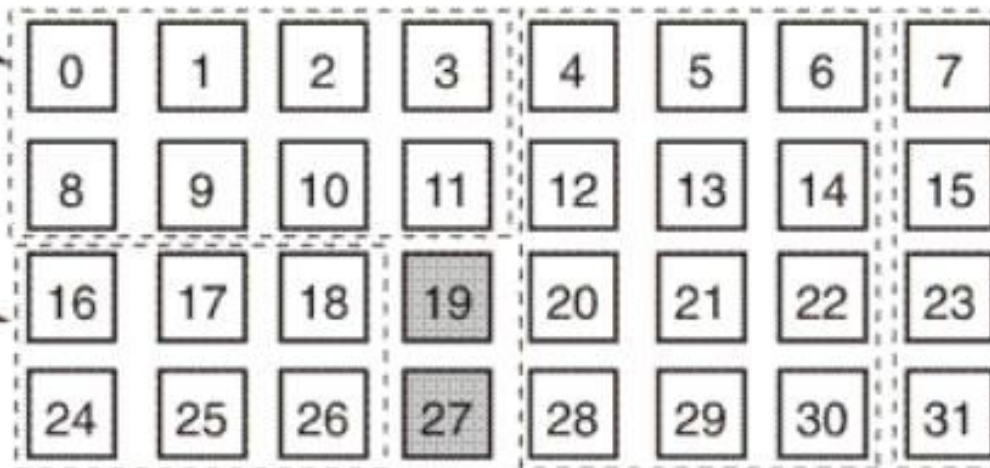
Partição em 8 CPUs

Partição em 4 CPUs

Partição em 6 CPUs

CPU não-alocada

Partição em 12 CPUs



Afinidade de processador: procura manter um processo na mesma CPU em que ele já estava executando. Isso é vantajoso porque aumenta a chance de os dados ainda estarem no cache daquela CPU, reduzindo o custo de mover o processo entre processadores.

Balanceamento de carga: distribui os processos entre os vários processadores para evitar que uma CPU fique sobrecarregada enquanto outra esteja ociosa. Pode ocorrer de duas formas: push migration, quando o sistema move processos de uma CPU muito ocupada para outra menos ocupada, e pull migration, quando uma CPU ociosa busca processos de outra CPU.

Escalonamento assimétrico: um processador principal fica responsável pelas decisões de escalonamento e pela distribuição das tarefas, enquanto os demais apenas executam os processos atribuídos. É simples, mas pode criar um gargalo no processador principal.

Escalonamento simétrico – SMP (Symmetric Multiprocessing): todos os processadores podem participar do escalonamento. Os processos podem ser retirados de uma fila comum ou de filas específicas por CPU. É mais flexível e distribui melhor o trabalho, mas exige sincronização entre os processadores.

Exercicio

A tabela a seguir representa um conjunto de tarefas prontas para utilizar um processador:

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	3	5	7
duração	5	4	5	6	4
prioridade	2	3	5	9	6

Represente graficamente a sequência de execução das tarefas e calcule os tempos médios de vida (*turnaround time*) e de espera (*waiting time*), para as políticas de escalonamento a seguir:

- (a) FCFS cooperativa
- (b) SJF cooperativa
- (c) SJF preemptiva (SRTF)
- (d) PRIO cooperativa
- (e) PRIO preemptiva
- (f) RR com $t_q = 2$, sem envelhecimento

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

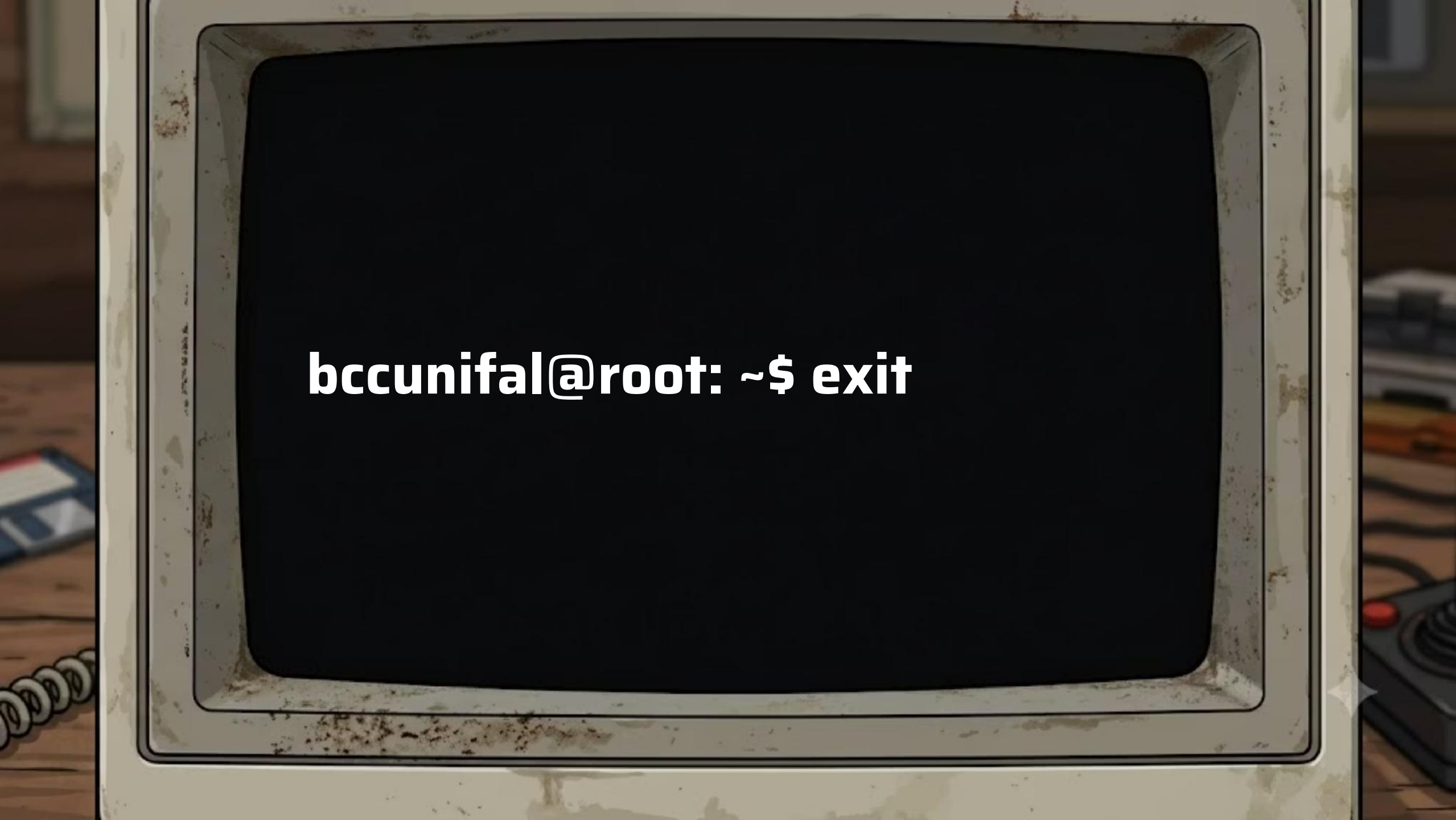
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit